



提供所提方法的模型框架，现在一个基本的HGNN作为 **backbone**，基于上述5个维度，基于现有的开源的超图数据集，借助DHG library支持的多个领域的数据集，给出模型实现的细节描述，以及模型在不同任务上的效果提升，先提供一个跨领域的效果提升的测试代码的要求，主要这些模型和实验描述必须清晰，阐述到位，需要让另一个agent可以基于这个描述给出正确的实现。

可以。下面我给你一份可直接交给另一个 agent 实现的模型与实验说明，目标是：基于一个基本HGNN backbone，做一个插件式的 **unified hypergraph structural encoder**，围绕你确定的主视角“node-hyperedge-motif-community-global-cross-hypergraph”，同时覆盖你前面提出的 5 类结构信息；并基于 DHG / DeepHypergraph 及其支持的数据集组织一个跨领域预训练与迁移评估方案。

[1] [2] [3] [4] [5]

我会分成 5 部分：

- 模型框架
- 任务设计与损失
- 数据与实验协议
- 效果提升预期与报告方式
- 交给实现 agent 的代码要求

1. 模型框架

这个方法应被实现为一个插件式预训练 **encoder**，而不是一个完全绑定单任务的 end-to-end 模型。DHG 已经提供了多种超图模型，包括 HGNN、HyperGCN、HNHN、UniGCN、UniGAT、UniSAGE、UniGIN 等，因此最稳妥的起点是先用 HGNN 或 UniGCN 作为 **backbone**，再在其上加多任务预训练头。 [2] [5]

1.1 总体定位

- 输入：一组来自多个领域的超图 $\{\mathcal{H}^{(d)}\}_{d=1}^D$ ，每个超图可写为 $\mathcal{H} = (V, E, X)$ ，其中 V 是节点集， E 是超边集， X 是节点特征。
- 输出：
 - 节点表示 $Z_v \in \mathbb{R}^{n \times h}$
 - 超边表示 $Z_e \in \mathbb{R}^{m \times h}$

- 子超图 / motif 表示 Z_s
- 全图表示 Z_g
- 使用方式：
 - 预训练阶段：在多域超图上优化多任务 loss；
 - 下游阶段：将 encoder 参数加载到任意 DHG 支持的 HGNN backbone 中，再接节点分类、超边预测、图分类等 task head 微调。

这种“插件式 encoder”定位，与 Hyper-FM 的多域预训练目标一致，但你这里不绑定文本属性和其专用层次模块，而是强调结构多尺度统一编码；也与 HGFM 的“先预训练 encoder，再做少样本微调”思路一致，但不局限于脑网络。 [6] [7] [8]

1.2 Backbone 设计

建议第一版 backbone 采用两层 HGNN 风格编码器：

$$H^{(1)} = \sigma(\text{HGNNConv}(X, \mathcal{H}))$$

$$H^{(2)} = \text{HGNNConv}(H^{(1)}, \mathcal{H})$$

其中 $H^{(2)}$ 就是节点级结构表示 Z_v 。 [2]

实现细节建议：

- 默认 backbone：dhg.models.HGNN 或基于 dhg.nn 的自定义两层 HGNNConv。 [9] [2]
- hidden dim：128 或 256
- dropout：0.3-0.5
- 激活：ReLU
- 残差：建议加 shallow residual，避免预训练阶段过平滑
- 可选替换 backbone：UniGCN / HNHN，用于 backbone-agnostic 实验。 [2]

1.3 多粒度表示读取

为了支持 node / hyperedge / motif / community / global / cross-hypergraph 六层任务，需要定义多种 readout：

- 节点表示： $Z_v = H^{(2)}$
- 超边表示：对超边内节点做 pooling

$$z_e = \text{Pool}(\{z_v \mid v \in e\})$$

推荐 mean + max concat

- motif / 子超图表示：对采样子超图中的节点和超边联合 pooling
- community 表示：对某个社区中的节点做 pooling
- 全图表示：对全图节点表示做 mean pooling 或 attention pooling

$$z_g = \text{Readout}(Z_v)$$

2. 任务设计与损失

主视角按你确定的结构层级来组织：

node $\rightarrow$ **hyperedge** $\rightarrow$ **motif** $\rightarrow$ **community** $\rightarrow$ **global** $\rightarrow$ **cross-hypergraph**。

为了控制复杂度，第一版建议实现一个 最小闭环 5-6 任务集，对应每个关键层级各一个任务。

2.1 Node-level：高阶邻居模式预测

目标：让节点表示编码节点度、高阶邻域和结构角色。

这和 Hyper-FM 中“高阶邻居向量 + 层次聚类 + 自监督预测”的思想相近，但这里不依赖文本语义，纯结构实现即可。 [8] [6]

构造方式

1. 对每个节点 v ，基于 incidence 结构构造高阶邻居特征向量：
 - 节点度
 - 所属超边尺寸统计 (mean / max / histogram)
 - 2-hop 可达节点数
 - overlap-based 邻居统计
2. 对所有节点的该特征做 KMeans / hierarchical clustering，得到 pseudo label c_v
3. 用节点表示 z_v 预测 cluster id

损失

$$\mathcal{L}_{node} = \text{CE}(f_{node}(z_v), c_v)$$

作用

- 对应结构信息：节点度及高阶邻居
- 对应层级：node

2.2 Hyperedge-level：超边存在性 / completion

目标：建模超边内部的共现模式和高阶关系。

这类任务已经被 GENET、PhyGCN 等证明是有效的超图预训练目标。 [10] [11]

构造方式

- 正样本：真实超边 $e \in E$
- 负样本：从真实超边随机替换若干节点得到伪超边 $\tilde{e}$
- 或 mask 超边中的一个节点，做 hyperedge completion

表示

$$z_e = \text{Pool}(\{z_v \mid v \in e\})$$

损失

二分类版本：

$$\mathcal{L}_{edge} = - \sum_{e \in E^+ \cup E^-} y_e \log \hat{y}_e + (1 - y_e) \log(1 - \hat{y}_e)$$

completion 版本：

$$\mathcal{L}_{comp} = \text{CE}(f_{comp}(z_{e \setminus v}), v)$$

作用

- 对应结构信息：超边尺寸与共现
- 对应层级：hyperedge

2.3 Motif-level：局部高阶子结构分类 / 对比

目标：显式学习超边重叠模式和 incidence pattern。

这是你相对 Hyper-FM/HGFM 可以更突出的 novel 设计，因为现有两者都没有把 motif-level task 明确抽出来。 [17] [6]

构造方式

- 采样小型子超图（例如基于一个 seed hyperedge 扩展 1-hop overlap）
- 为子超图计算结构签名：
 - 节点数、超边数
 - 平均超边尺寸
 - overlap density
 - incidence bipartite clustering coefficient
- 对这些结构签名做聚类得到 motif type label c_s

损失

分类版本：

$$\mathcal{L}_{motif} = \text{CE}(f_{motif}(z_s), c_s)$$

对比版本：

$$\mathcal{L}_{motif_con} = \text{InfoNCE}(z_s^{(1)}, z_s^{(2)}, \{z_{s^-}\})$$

作用

- 对应结构信息：incidence pattern + 局部高阶结构
- 对应层级：motif

2.4 Community-level：社区伪标签预测

目标：让 encoder 感知中观结构。

GFSE 在图上用 community detection 作为中观任务，你在超图上完全可以对应做 hypergraph community pseudo-supervision。

构造方式

- 基于 hypergraph Laplacian / clique expansion / incidence bipartite graph 做社区检测
- 给节点或超边分配 community label c^{comm}

损失

节点级社区预测：

$$\mathcal{L}_{comm} = \text{CE}(f_{comm}(z_v), c_v^{comm})$$

可加 community contrastive：

$$\mathcal{L}_{intra/inter}$$

让同社区节点更近、异社区节点更远

作用

- 对应结构信息：局部-全局之间的 mesoscopic 结构
- 对应层级：community

2.5 Global-level：全图对比学习

目标：学习全局拓扑统计和鲁棒图级表示。

GENET 已经显示 local/global contrastive 在超图预训练中有效。 [10]

构造方式

- 对同一超图生成两个 augment view：
 - 节点 dropout
 - 超边 dropout
 - incidence perturbation
- 编码得到两个图表示 $z_g^{(1)}, z_g^{(2)}$

损失

$$\mathcal{L}_{global} = \text{InfoNCE}(z_g^{(1)}, z_g^{(2)}, \{z_{g^-}\})$$

作用

- 对应结构信息：global topology
- 对应层级：global

2.6 Cross-hypergraph-level：跨域子超图原型对齐

目标：学习 domain-invariant 的结构表示。

这部分与 Hyper-FM 的多超图结构知识抽取最接近，但你这里更明确地基于 motif/community 子结构和结构原型来做。 [6] [8]

构造方式

1. 从所有领域超图中采样 motif/community 子超图
2. 计算结构统计特征并聚类，得到 prototype id p_s
3. 用 encoder 输出的子超图表示预测 prototype，或做 prototype contrastive

损失

分类版本：

$$\mathcal{L}_{cross} = \text{CE}(f_{proto}(z_s), p_s)$$

或 prototype contrastive：

$$\mathcal{L}_{proto} = -\log \frac{\exp(\text{sim}(z_s, q_{p_s})/\tau)}{\sum_j \exp(\text{sim}(z_s, q_j)/\tau)}$$

作用

- 对应结构信息：多超图/多域结构
- 对应层级：cross-hypergraph

2.7 总损失

建议第一版总目标：

$$\mathcal{L} = \lambda_1 \mathcal{L}_{node} + \lambda_2 \mathcal{L}_{edge} + \lambda_3 \mathcal{L}_{motif} + \lambda_4 \mathcal{L}_{comm} + \lambda_5 \mathcal{L}_{global} + \lambda_6 \mathcal{L}_{cross}$$

初始可设：

- $\lambda_1 = \lambda_2 = \lambda_3 = \lambda_4 = 1$
- $\lambda_5 = 0.5$

- $\lambda_6 = 1$

3. 数据与实验协议

3.1 数据来源

DHG / DeepHypergraph 是一个支持图和超图学习的 PyTorch 库，内置多种超图模型和多类数据集。

[3] [4] [5] [1]

可优先使用 DHG 自带或便于转为 DHG 格式的数据：

- 引文 / 共引 / 合著类：cora, citeseer, cocitation, coauthorship 等模块可用。 [4]
- 内容 / 文档类：cooking_200 等。 [4]
- 再补充若干来自 DHG-Bench 的多任务数据，因其覆盖 node-, edge-, graph-level 三类任务，并在 22 个多样化超图数据集上统一评估 17 个 HNN，适合做更系统 benchmark。 [12] [13] [14]

如果你要突出“跨领域”，建议把数据组织成 4 个 domain bucket：

Domain	示例
Citation/Academic	Cora, Citeseer, Coauthorship, Cocitation
Content/Document	Cooking / 文档属性超图
Recommendation / Interaction	若有可转 DHG 格式的数据
Biological / Other	DHG-Bench 或外部公开超图

3.2 预训练协议

- 在多个领域的超图上联合预训练 encoder
- 采样方式：
 - 每个 batch 从不同 domain 均匀采样 1-k 个超图/子超图
 - 保证 cross-domain task 中有跨领域样本混合
- 训练轮数：100–300 epochs
- optimizer：Adam
- lr：1e-3
- weight decay：1e-5
- early stopping：基于 held-out pretext loss 或 downstream validation

3.3 下游任务

至少覆盖 3 类任务，这样与 DHG-Bench 的 node/edge/graph 三层 benchmark 叙事一致。 [13] [12]

1. 节点分类

- 冻结 encoder + linear probe
- 全量微调

2. 超边/链接预测

- 用超边表示做二分类

3. 图/超图分类

- 用全图 readout 做分类

3.4 跨领域迁移测试

这是你现在最需要的“先提供代码要求”的重点。

设定 A : Leave-one-domain-out pretraining

- 在 $D - 1$ 个领域上预训练
- 在 held-out domain 上做 few-shot / full-shot 微调
- 观察：
 - scratch HGNN
 - single-domain pretrain
 - multi-domain pretrain (你的方法)

设定 B : Cross-task transfer

- 在 node-level 丰富的领域上预训练
- 迁移到 edge-level / graph-level 任务
- 观察多任务预训练是否比单任务预训练更稳

4. 效果提升预期与报告方式

你现在先不需要写死数字，但需要给实现 agent 一个合理的结果预期框架。

4.1 可参考的已有收益

- Hyper-FM 在多域文本属性超图上平均提升约 13.3%–13.4%。^{[15] [8]}
- HGFM 在脑疾病诊断任务上全面优于 SOTA，说明预训练对高阶结构任务有明显收益。^[7]
- DHG-Bench 的存在也说明社区已经开始系统比较 effectiveness / efficiency / robustness / fairness，这种基准化评估是合理方向。^{[12] [13]}

4.2 你的方法预期收益应如何表述

建议在描述中给实现 agent 明确要求：报告时至少比较下面三类改进：

- vs scratch backbone：
 - 预期在 node/edge/graph 三类任务上有稳定提升
- vs 单任务预训练：
 - 多任务预训练应在跨域迁移上更稳

- **vs 单域预训练：**
 - 多域预训练应在 held-out domain 上更强，尤其 few-shot 情况下更明显

4.3 报告表格建议

实现 agent 应输出至少 3 张主表：

1. In-domain downstream performance

2. Cross-domain transfer performance

3. Ablation on tasks

- 去掉 node / edge / motif / community / global / cross 中任一项

以及 2 张分析图：

- 预训练 domain 数量 vs downstream 性能
- 不同任务组合 vs held-out domain 性能

5. 交给实现 agent 的代码要求

下面这部分可以直接作为“实现 specification”。

5.1 项目结构

```
project/  
  configs/  
    pretrain.yaml  
    finetune_node.yaml  
    finetune_edge.yaml  
    finetune_graph.yaml  
  data/  
    dhg/  
    external/  
  models/  
    backbone.py  
    encoder.py  
    heads.py  
    pretext_tasks.py  
  trainers/  
    pretrain_trainer.py  
    finetune_trainer.py  
  utils/  
    sampling.py  
    clustering.py  
    eval.py  
    metrics.py  
  scripts/  
    run_pretrain.py  
    run_transfer.py  
    run_ablation.py
```

5.2 最低可实现版本

先实现：

- backbone: HGNN
- tasks:
 - node pattern prediction
 - hyperedge prediction
 - motif classification
 - global contrastive
 - cross-hypergraph prototype alignment

community task 可以先作为第二阶段加入。

也就是说第一版最小闭环是 5 个任务。

5.3 输入输出接口要求

Encoder

```
node_emb, edge_emb, graph_emb, aux = encoder(hg, x)
```

其中：

- hg: DHG hypergraph object
- x: node features
- node_emb: [num_nodes, hidden_dim]
- edge_emb: [num_edges, hidden_dim]
- graph_emb: [hidden_dim]
- aux: motif/subgraph embeddings and masks

Pretraining task heads

```
loss_dict = {  
    "node": ...,  
    "edge": ...,  
    "motif": ...,  
    "global": ...,  
    "cross": ...,  
    "total": ...  
}
```

5.4 跨领域测试代码要求

实现 agent 必须支持下面的实验入口：

```
python scripts/run_pretrain.py --config configs/pretrain.yaml
python scripts/run_transfer.py --config configs/finetune_node.yaml --heldout_domain (
python scripts/run_transfer.py --config configs/finetune_edge.yaml --heldout_domain (
python scripts/run_ablation.py --config configs/pretrain.yaml --drop_task motif
```

5.5 日志与结果要求

必须保存：

- 每个任务 loss 曲线
- 每个 domain 的预训练采样数
- 下游任务平均值和标准差（至少 3 seeds）
- 迁移实验中的 held-out domain 指标
- ablation 表格 csv

5.6 实现中必须明确的细节

实现 agent 不能自行猜测，必须按下列约定：

- 节点模式伪标签：先离线聚类生成，再训练
- motif 采样：固定采样 budget，例如每图每 epoch 采样 500 个子超图
- global contrastive augment：超边 dropout + 节点 feature masking
- cross-hypergraph prototypes：每 N 个 epoch 更新一次聚类中心
- edge negative sampling：按超边尺寸匹配负采样
- 所有实验固定 3 seeds
- 所有对比 baseline 使用同一个 backbone hidden dim

最后建议

如果你的目标是“让另一个 agent 能正确实现”，最重要的是不要一开始就把任务数做得太满。

第一版推荐的可执行组合是：

- Node pattern prediction
- Hyperedge prediction
- Motif classification
- Global contrastive
- Cross-hypergraph prototype alignment

这是一个已经具备：

- node
- hyperedge
- motif
- global
- cross-hypergraph

五层闭环的版本，同时覆盖了你之前定义的 5 类结构信息中的绝大多数关键部分。等第一版跑通后，再加 community task。

从叙事上，这个方案和 GFSE 类似，都是“多尺度结构预训练 + 插件式 encoder”；但你的工作建立在**超图原生对象 (hyperedge, incidence, motif overlap) **上，而不是 graph 的直接替换，这正是实现和论文里都需要反复强调的区别。^{[16] [8] [6] [7]}

如果你要，我下一步可以继续直接帮你写一版***“给另一个 agent 的实现说明文档”***，用更工程化的格式输出，例如：

- Objective
- Modules
- APIs
- Data assumptions
- Training loop
- Evaluation loop
- Failure cases
- Sanity checks

这样会更像真正的 implementation brief。

✱

1. <https://github.com/iMoonLab/DeepHypergraph/blob/main/docs/source/index.rst>
2. <https://deephypergraph.readthedocs.io/en/latest/api/models.html>
3. <https://deephypergraph.readthedocs.io/en/0.9.2/index.html>
4. <https://deephypergraph.readthedocs.io/en/latest/modules/index.html>
5. <https://github.com/iMoonLab/DeepHypergraph>
6. <https://www.themoonlight.io/zh/review/hypergraph-foundation-model>
7. <https://pubmed.ncbi.nlm.nih.gov/40244841/>
8. <https://arxiv.org/abs/2503.01203>
9. <https://deephypergraph.readthedocs.io/en/latest/api/nn.html>
10. <https://arxiv.org/html/2311.13121v2>
11. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10592843/>
12. <https://arxiv.org/abs/2508.12244v2>
13. <https://libraries.io/pypi/dhg-bench>

14. <https://arxiv.org/html/2508.12244v1>
15. <https://pubmed.ncbi.nlm.nih.gov/41433164/>
16. <https://jos.org.cn/html/2022/2/6353.htm>
17. <https://www.themoonlight.io/en/review/hypergraph-foundation-model>
18. <https://github.com/iMoonLab/DeepHypergraph/blob/main/docs/source/tutorial/model.rst>
19. <http://arxiv.org/abs/2506.12362>